Import of libraries and upload data file. Used a more generic format of upload as I had issues at first with figuring out the path. This method asks for the data file to be used for the rest of the code cells.

```
import pandas as pd
import numpy as np
import re
from google.colab import files

uploaded = files.upload()                     # choose Messy_data.csv
fname = list(uploaded.keys())[0]              # works even if Colab renames the file

# dtype=str keeps every value as raw text, so nothing converts silently
raw = pd.read_csv(fname, dtype=str)
print("Loaded:", fname, raw.shape)
print(raw.head().to_string())
```

```
Choose Files Messy_data.csv
Messy_data.csv(text/csv) - 13066 bytes, last modified: 11/1/2026 - 100% done
Saving Messy_data.csv to Messy_data (3).csv
Loaded: Messy_data (3).csv (185, 9)
  OrderID  CustomerName     City     Product Quantity UnitPrice   OrderDate OrderStatus CustomerAge
0   10001   Nikhil Jain   Mumbai      Laptop      3.0  50790.72  2026-01-18   Completed        61.0
1   10002   Simran Kaur   delhi      Monitor      1.0  10307.27  2026-01-28     Pending        50.0
2   10003      Zoya Ali    Delhi  Headphones      5.0   2439.64  2026-02-27   Delivered        35.0
3   10004  Priya Bansal    Delhi      Tablet      4.0  20945.65  02/03/2026     Pending        66.0
4   10005           NaN   Mumbai       Phone      4.0  15822.27  2026-02-14   Delivered        34.0
```

Data review to understand all types of data within the columns that are considered "*messy*." I call this my **Quality Audit**.

```
audit = raw.copy()

print("Shape:", audit.shape)
print(audit.head().to_string())

print("\n--- info() ---")
audit.info()

print("\n--- Missing values per column ---")
print(audit.isnull().sum())

print("\n--- Exact duplicate rows ---")
print(audit.duplicated().sum(), "(the brief says to expect 5)")

print("\n--- City spellings ---")
print(audit["City"].value_counts(dropna=False))

print("\n--- Status spellings ---")
print(audit["OrderStatus"].value_counts(dropna=False))

for col in ["Quantity", "UnitPrice", "CustomerAge"]:
    test = pd.to_numeric(audit[col], errors="coerce")
    bad = audit.loc[test.isna() & audit[col].notna(), ["OrderID", col]]
    print(f"\nNon-numeric values in {col}:")
    print(bad.to_string(index=False))

iso_test = pd.to_datetime(audit["OrderDate"], format="%Y-%m-%d", errors="coerce")
print("\nNon-ISO or invalid dates:")
print(audit.loc[iso_test.isna() & audit["OrderDate"].notna(), ["OrderID", "OrderDate"]].to_string(index=False))
```

```
Pune           21
Noida          21
Chennai        20
Gurugram       18
NaN             2
 delhi          1
MUMBAI          1
pUnE            1
  bengaluru     1
Name: count, dtype: int64

--- Status spellings ---
OrderStatus
Delivered     42
Shipped       38
Completed     34
Pending       34
Cancelled     32
NaN            2
completed      1
SHIPPED        1
 pending       1
Name: count, dtype: int64

Non-numeric values in Quantity:
OrderID Quantity
  10008    three
  10034    2 pcs

Non-numeric values in UnitPrice:
OrderID UnitPrice
  10016    18,500
  10079   unknown

Non-numeric values in CustomerAge:
OrderID CustomerAge
  10055       forty
  10113    29 years

Non-ISO or invalid dates:
OrderID  OrderDate
  10004 02/03/2026
  10026 15-02-2026
  10099 not a date
  10026 15-02-2026
```

Focus on standardization of text and rebuild the df.

```
df = raw.copy()    # fresh start every time this cell runs

# Order matters: strip first, then fix capitalization
for c in ["CustomerName", "City", "Product", "OrderStatus"]:
    df[c] = (df[c].str.strip()
                  .str.replace(r"\s+", " ", regex=True)
                  .str.title())

# Known variant spellings
df["City"] = df["City"].replace({"Bombay": "Mumbai",
                                 "Bangalore": "Bengaluru",
                                 "Gurgaon": "Gurugram"})

print(df["City"].value_counts(dropna=False))
print(df["OrderStatus"].value_counts(dropna=False))
```

```
City
Bengaluru    34
Delhi        24
Mumbai       23
Pune         22
Kolkata      21
Noida        21
Chennai      20
Gurugram     18
NaN           2
Name: count, dtype: int64
OrderStatus
Delivered    42
Shipped      39
Pending      35
Completed    35
Cancelled    32
```

```
NaN           2
Name: count, dtype: int64
```

Fix the data types.

```
WORDS = {"zero":0,"one":1,"two":2,"three":3,"four":4,"five":5,"six":6,"seven":7,
         "eight":8,"nine":9,"ten":10,"twenty":20,"thirty":30,"forty":40,"fifty":50,
         "sixty":60,"seventy":70,"eighty":80,"ninety":90}

def to_number(x):
    """'three'->3, '2 pcs'->2, '18,500'->18500, 'unknown'->NaN"""
    if pd.isna(x):
        return np.nan
    s = str(x).strip().lower().replace(",", "").replace("₹", "").replace("$", "")
    if s in WORDS:
        return float(WORDS[s])
    m = re.search(r"-?\d+(\.\d+)?", s)
    return float(m.group()) if m else np.nan

# Convert numbers and show exactly what changed (audit trail)
for col in ["Quantity", "UnitPrice", "CustomerAge"]:
    before = df[col].copy()
    df[col] = before.apply(to_number).astype(float)
    plain = pd.to_numeric(before, errors="coerce")
    changed = before.notna() & (plain.isna() | (plain != df[col]))
    print(f"\n{col}: values that needed special handling")
    print(pd.DataFrame({"OrderID": df.loc[changed, "OrderID"],
                        "original": before[changed],
                        "new": df.loc[changed, col]}).to_string(index=False))

# Dates: ISO format first, then day-first for the rest (e.g. 15-02-2026)
iso = pd.to_datetime(df["OrderDate"], format="%Y-%m-%d", errors="coerce")
rest = df["OrderDate"].where(iso.isna())
other = rest.apply(lambda s: pd.NaT if pd.isna(s)
                   else pd.to_datetime(s, dayfirst=True, errors="coerce"))
df["OrderDate"] = pd.to_datetime(iso.fillna(other))

df["OrderID"] = pd.to_numeric(df["OrderID"], errors="coerce").astype("Int64")

print("\n--- Data types now ---")
print(df.dtypes)
```

```
Quantity: values that needed special handling
OrderID original  new
  10008    three  3.0
  10034    2 pcs  2.0

UnitPrice: values that needed special handling
OrderID original     new
  10016   18,500 18500.0
  10079  unknown     NaN

CustomerAge: values that needed special handling
OrderID original  new
  10055    forty 40.0
  10113 29 years 29.0

--- Data types now ---
OrderID                  Int64
CustomerName            object
City                    object
Product                 object
Quantity               float64
UnitPrice              float64
OrderDate       datetime64[ns]
OrderStatus             object
CustomerAge            float64
dtype: object
```

Apply the Business Rules based on the Project requests.

```
today = pd.Timestamp.today().normalize()

# .notna() guards stop missing values from being flagged as violations
rules = {
```

```
    "Quantity < 1 or non-whole": df["Quantity"].notna() & ((df["Quantity"] < 1) | (df["Quantity"] % 1 != 0)),
    "UnitPrice <= 0":            df["UnitPrice"].notna() & (df["UnitPrice"] <= 0),
    "Age outside 0-120":         df["CustomerAge"].notna() & ((df["CustomerAge"] < 0) | (df["CustomerAge"] > 120)),
    "Placeholder or implausible price": df["UnitPrice"].notna() & ((df["UnitPrice"] >= 999999) | (df["UnitPrice"] < 1)),
    "Future OrderDate":          df["OrderDate"].notna() & (df["OrderDate"] > today),
}

for name, mask in rules.items():
    print(f"{name}: {mask.sum()} rows")
    if mask.sum():
        print(df.loc[mask, ["OrderID", "Quantity", "UnitPrice", "CustomerAge", "OrderDate"]].to_string(index=False))
    print()

# Invalid values become missing so they can't distort medians later
df.loc[rules["Quantity < 1 or non-whole"], "Quantity"] = np.nan
df.loc[rules["UnitPrice <= 0"], "UnitPrice"] = np.nan
df.loc[rules["Age outside 0-120"], "CustomerAge"] = np.nan
df.loc[rules["Placeholder or implausible price"], "UnitPrice"] = np.nan
df.loc[rules["Future OrderDate"], "OrderDate"] = pd.NaT
```

```
Quantity < 1 or non-whole: 1 rows
 OrderID  Quantity  UnitPrice  CustomerAge  OrderDate
   10068      -3.0    7114.26         53.0 2026-03-06

UnitPrice <= 0: 0 rows

Age outside 0-120: 1 rows
 OrderID  Quantity  UnitPrice  CustomerAge  OrderDate
   10049       2.0    24406.4        185.0 2026-01-23

Placeholder or implausible price: 2 rows
 OrderID  Quantity  UnitPrice  CustomerAge  OrderDate
   10023       4.0  999999.00         46.0 2026-03-25
   10093       4.0       0.01         32.0 2026-01-12

Future OrderDate: 0 rows
```

Remove duplicates.

```
exact = df.duplicated().sum()
print("Exact duplicate rows:", exact)

dup_ids = df[df["OrderID"].duplicated(keep=False)].sort_values("OrderID")
print("Rows sharing an OrderID:", len(dup_ids))
if len(dup_ids):
    print(dup_ids.to_string()) # inspect before deleting

rows_before = len(df)
df = df.drop_duplicates().drop_duplicates(subset="OrderID", keep="first").reset_index(drop=True)
removed = rows_before - len(df)
print(f"Removed {removed} duplicate rows -> {len(df)} remain")
print("Matches the brief's 5 duplicates ✅" if removed == 5
      else f"⚠️ Brief says 5 but {removed} were removed. Review the table above.")
```

```
Exact duplicate rows: 5
Rows sharing an OrderID: 10
     OrderID  CustomerName       City     Product  Quantity  UnitPrice  OrderDate OrderStatus  CustomerAge
5      10006  Priya Bansal      Delhi  Smartwatch       5.0    5768.41 2026-02-18   Completed         53.0
180    10006  Priya Bansal      Delhi  Smartwatch       5.0    5768.41 2026-02-18   Completed         53.0
181    10026   Kabir Singh      Delhi    Keyboard       1.0    1506.37 2026-02-15   Cancelled         31.0
25     10026   Kabir Singh      Delhi    Keyboard       1.0    1506.37 2026-02-15   Cancelled         31.0
182    10081  Aarav Sharma  Bengaluru  Headphones       1.0    2591.00 2026-01-31     Pending         48.0
80     10081  Aarav Sharma  Bengaluru  Headphones       1.0    2591.00 2026-01-31     Pending         48.0
120    10121    Neha Gupta       Pune  Smartwatch       3.0    6517.30 2026-02-04   Cancelled         48.0
183    10121    Neha Gupta       Pune  Smartwatch       3.0    6517.30 2026-02-04   Cancelled         48.0
150    10151      Isha Roy     Mumbai  Smartwatch       3.0    6791.02 2026-01-15     Pending         20.0
184    10151      Isha Roy     Mumbai  Smartwatch       3.0    6791.02 2026-01-15     Pending         20.0
Removed 5 duplicate rows -> 180 remain
Matches the brief's 5 duplicates ✅
```

Detect, Flag and Cap Outliers. I have been working with outliers for many years using Python so I used my own code from previous projects.

```
def cap_by_group(data, value_col, group_col):
```

```
def cap_by_group(data, value_col, group_col):
    """Flag IQR outliers within each group, then cap them to the fences."""
    g = data.groupby(group_col)[value_col]
    q1 = g.transform(lambda s: s.quantile(0.25))
    q3 = g.transform(lambda s: s.quantile(0.75))
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    flag = (data[value_col] < low) | (data[value_col] > high)
    capped = data[value_col].clip(lower=low, upper=high)
    return flag, capped

# Prices: compared within each product (a laptop isn't an outlier among mice)
df["UnitPrice_outlier"], price_capped = cap_by_group(df, "UnitPrice", "Product")
print("Price outliers flagged:", df["UnitPrice_outlier"].sum())
print(df.loc[df["UnitPrice_outlier"], ["OrderID", "Product", "UnitPrice"]]
        .assign(capped_to=price_capped[df["UnitPrice_outlier"]].round(2))
        .to_string())
df["UnitPrice"] = price_capped

# Quantity and Age: single-column IQR
def iqr_flag_cap(s):
    q1, q3 = s.quantile([0.25, 0.75])
    i = q3 - q1
    low, high = q1 - 1.5 * i, q3 + 1.5 * i
    return (s < low) | (s > high), s.clip(low, high)

df["Quantity_outlier"], qty_capped = iqr_flag_cap(df["Quantity"])
df["Age_outlier"], age_capped = iqr_flag_cap(df["CustomerAge"])
df["Quantity"], df["CustomerAge"] = qty_capped, age_capped

print(df[["Quantity_outlier", "Age_outlier"]].sum())
```

```
Price outliers flagged: 4
     OrderID     Product  UnitPrice  capped_to
6      10007  Headphones    2159.37    2169.07
15     10016      Laptop   18500.00   37801.00
109    10110      Laptop     709.84   37801.00
140    10141       Phone     815.82   11601.91
Quantity_outlier    1
Age_outlier         0
dtype: int64
```

Handling missing values.

```
print("Missing BEFORE:")
print(df.isnull().sum(), "\n")

# Flag first so the original gaps stay visible
for c in ["Quantity", "UnitPrice", "CustomerAge"]:
    df[f"{c}_imputed"] = df[c].isna()

# Numbers: median (robust to outliers). Price uses its product's median.
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median()).round().astype(int)
df["UnitPrice"] = df["UnitPrice"].fillna(df.groupby("Product")["UnitPrice"].transform("median"))
df["UnitPrice"] = df["UnitPrice"].fillna(df["UnitPrice"].median()).round(2)
df["CustomerAge"] = df["CustomerAge"].fillna(df["CustomerAge"].median()).round().astype(int)

# Labels: honest "Unknown" instead of guessing
for c in ["CustomerName", "City", "Product", "OrderStatus"]:
    df[c] = df[c].fillna("Unknown")

# Dates stay missing on purpose: inventing a date would be fabricating data
print("Missing AFTER:")
print(df.isnull().sum())
```

```
Missing BEFORE:
OrderID              0
CustomerName         2
City                 2
Product              0
Quantity             3
UnitPrice            5
OrderDate            3
OrderStatus          2
CustomerAge          3
UnitPrice_outlier    0
```

```
Quantity_outlier     0
Age_outlier          0
dtype: int64

Missing AFTER:
OrderID                0
CustomerName           0
City                   0
Product                0
Quantity               0
UnitPrice              0
OrderDate              3
OrderStatus            0
CustomerAge            0
UnitPrice_outlier      0
Quantity_outlier       0
Age_outlier            0
Quantity_imputed       0
UnitPrice_imputed      0
CustomerAge_imputed    0
dtype: int64
```

Quality assurance check. Ensure all features from what was discovered in Cell 2 where we identified all the "*messy*" data.

```
df["TotalAmount"] = (df["Quantity"] * df["UnitPrice"]).round(2)
df["UnitPrice_log"] = np.log1p(df["UnitPrice"]).round(4)    # optional skew reduction

assert df["OrderID"].is_unique
assert (df["Quantity"] >= 1).all()
assert (df["UnitPrice"] > 0).all()
assert df["CustomerAge"].between(0, 120).all()
assert df["City"].eq(df["City"].str.strip()).all()
print("All checks passed ✅\n")

print("Shape:", df.shape, "\n")
df.info()
print()
print(df.isnull().sum(), "\n")
print(df["City"].value_counts(), "\n")
print(df["OrderStatus"].value_counts())
print(df.head().to_string())
```

```
 8   CustomerAge          180 non-null    int64
 9   UnitPrice_outlier    180 non-null    bool
 10  Quantity_outlier     180 non-null    bool
 11  Age_outlier          180 non-null    bool
 12  Quantity_imputed     180 non-null    bool
 13  UnitPrice_imputed    180 non-null    bool
 14  CustomerAge_imputed  180 non-null    bool
 15  TotalAmount          180 non-null    float64
 16  UnitPrice_log        180 non-null    float64
dtypes: Int64(1), bool(6), datetime64[ns](1), float64(3), int64(2), object(4)
memory usage: 16.8+ KB

OrderID                0
CustomerName           0
City                   0
Product                0
Quantity               0
UnitPrice              0
OrderDate              3
OrderStatus            0
CustomerAge            0
UnitPrice_outlier      0
```

```
Gurugram     18
Unknown       2
Name: count, dtype: int64

OrderStatus
Delivered    42
Shipped      39
Completed    34
Pending      33
Cancelled    30
Unknown       2
Name: count, dtype: int64
   OrderID  CustomerName    City     Product  Quantity  UnitPrice  OrderDate OrderStatus  CustomerAge  UnitPrice_outlier  Quant
0    10001   Nikhil Jain  Mumbai      Laptop         3   50790.72 2026-01-18   Completed           61              False
1    10002   Simran Kaur   Delhi     Monitor         1   10307.27 2026-01-28     Pending           50              False
2    10003      Zoya Ali   Delhi  Headphones         5    2439.64 2026-02-27   Delivered           35              False
3    10004  Priya Bansal   Delhi      Tablet         4   20945.65 2026-03-02     Pending           66              False
4    10005       Unknown  Mumbai       Phone         4   15822.27 2026-02-14   Delivered           34              False
```

Merging of regions and calculating regional sales.

Gemini

```
regions = pd.DataFrame({
    "City": ["Delhi", "Noida", "Gurugram", "Mumbai", "Pune", "Bengaluru", "Ch
    "Region": ["North", "North", "North", "West", "West", "South", "South", "

demo = df.merge(regions, on="City", how="left")
demo["Region"] = demo["Region"].fillna("Unknown")
print(demo.groupby("Region")["TotalSales"].sum().round(2))

print(demo.groupby("Region")["TotalAmount"].sum().round(2))
```

```
Region
East       1426238.33
North      2018414.47
South      2266993.54
Unknown      76520.08
West       2068456.60
Name: TotalAmount, dtype: float64
```

Export the clean data to cleaned_sales_datasheet.csv file.

```
core = ["OrderID", "CustomerName", "City", "Product", "Quantity",
        "UnitPrice", "OrderDate", "OrderStatus", "CustomerAge", "TotalAmount"]
print(df[core].head().to_string())

# The submitted file: the 9 original columns plus TotalAmount
df[core].to_csv("cleaned_sales_dataset.csv", index=False)

# Optional audit file with the outlier/imputation flags
flags = ["UnitPrice_outlier", "Quantity_outlier", "Age_outlier",
         "Quantity_imputed", "UnitPrice_imputed", "CustomerAge_imputed", "UnitPrice_log"]
df[core + flags].to_csv("cleaned_sales_dataset_with_flags.csv", index=False)

files.download("cleaned_sales_dataset.csv")
files.download("cleaned_sales_dataset_with_flags.csv")
```

```
   OrderID  CustomerName    City     Product  Quantity  UnitPrice  OrderDate OrderStatus  CustomerAge  TotalAmount
0    10001   Nikhil Jain  Mumbai      Laptop         3   50790.72 2026-01-18   Completed           61    152372.16
1    10002   Simran Kaur   Delhi     Monitor         1   10307.27 2026-01-28     Pending           50     10307.27
2    10003      Zoya Ali   Delhi  Headphones         5    2439.64 2026-02-27   Delivered           35     12198.20
3    10004  Priya Bansal   Delhi      Tablet         4   20945.65 2026-03-02     Pending           66     83782.60
4    10005       Unknown  Mumbai       Phone         4   15822.27 2026-02-14   Delivered           34     63289.08
```

```
# ====================  CLEANING REPORT  ====================
cols = ["OrderID", "CustomerName", "City", "Product", "Quantity",
        "UnitPrice", "OrderDate", "OrderStatus", "CustomerAge"]
text_cols = ["CustomerName", "City", "Product", "OrderStatus"]
num_cols = ["Quantity", "UnitPrice", "CustomerAge"]

def header(title):
    print("\n" + "=" * 60)
    print(title)
    print("=" * 60)

# ---------- 1. Rows and duplicates ----------
```

```
header("1. ROWS AND DUPLICATES")
print(f"Raw rows:                {len(raw)}")
print(f"Clean rows:              {len(df)}")
print(f"Duplicate rows removed:  {len(raw) - len(df)}  (brief expects 5)")

# ---------- 2. Missing values ----------
header("2. MISSING VALUES (before vs after)")
missing = pd.DataFrame({"Missing before": raw[cols].isnull().sum(),
                        "Missing after":  df[cols].isnull().sum()})
print(missing.to_string())
print("\nNote: OrderDate stays missing on purpose (we never invent dates).")

# ---------- 3. Text problems ----------
header("3. TEXT PROBLEMS IN RAW DATA")
rows = []
for c in text_cols:
    s = raw[c]
    stripped = s.str.strip()
    rows.append({"Column": c,
                 "Extra spaces": int((s.notna() & (s != stripped)).sum()),
                 "Inconsistent capitals": int((s.notna() & (stripped != stripped.str.title())).sum())})
print(pd.DataFrame(rows).to_string(index=False))

print("\nDistinct City spellings:   raw",
      raw["City"].dropna().nunique(), "-> clean",
      df.loc[df["City"] != "Unknown", "City"].nunique())
print("Distinct Status spellings: raw",
      raw["OrderStatus"].dropna().nunique(), "-> clean",
      df.loc[df["OrderStatus"] != "Unknown", "OrderStatus"].nunique())

# ---------- 4. Type errors and business-rule violations ----------
header("4. TYPE ERRORS AND BUSINESS-RULE VIOLATIONS (raw data)")
rows = []
for c in num_cols:
    plain = pd.to_numeric(raw[c], errors="coerce")
    converted = raw[c].apply(to_number)
    text_type = int((plain.isna() & raw[c].notna()).sum())
    if c == "Quantity":
        rule = int((converted.notna() & ((converted < 1) | (converted % 1 != 0))).sum())
    elif c == "UnitPrice":
        rule = int((converted.notna() & (converted <= 0)).sum())
    else:
        rule = int((converted.notna() & ((converted < 0) | (converted > 120))).sum())
    rows.append({"Column": c,
                 "Stored as text": text_type,
                 "Rule violations": rule})
print(pd.DataFrame(rows).to_string(index=False))

prices_raw = raw["UnitPrice"].apply(to_number)
placeholder = int((prices_raw.notna() & ((prices_raw >= 999999) | (prices_raw < 1))).sum())
print(f"\nPlaceholder or implausible prices (>= 999999 or < 1): {placeholder}")

# ---------- 5. Dates ----------
header("5. DATE PROBLEMS")
iso = pd.to_datetime(raw["OrderDate"], format="%Y-%m-%d", errors="coerce")
non_iso = int((iso.isna() & raw["OrderDate"].notna()).sum())
other = raw["OrderDate"].where(iso.isna()).apply(
    lambda s: pd.NaT if pd.isna(s) else pd.to_datetime(s, dayfirst=True, errors="coerce"))
unreadable = int((iso.fillna(other).isna() & raw["OrderDate"].notna()).sum())
blank_dates = int(raw["OrderDate"].isna().sum())
print(f"Dates not in YYYY-MM-DD format:        {non_iso}")
print(f"  of those, unreadable (set to blank): {unreadable}")
print(f"Dates originally blank:                {blank_dates}")
print(f"Dates missing in clean data:           {int(df['OrderDate'].isna().sum())}")

# ---------- 6. Imputation ----------
header("6. VALUES IMPUTED (filled with a median)")
imp_rows = []
for c in num_cols:
    imputed = int(df[f"{c}_imputed"].sum())
    blank_originally = int(raw[c].isna().sum())
    imp_rows.append({"Column": c,
                     "Originally blank": blank_originally,
                     "Total imputed": imputed,
                     "Invalid, turned blank": max(imputed - blank_originally, 0)})
print(pd.DataFrame(imp_rows).to_string(index=False))
print("\n(Small differences can occur because the 5 duplicate rows were removed.)")
```

```
label_fill = {c: int((raw[c].isna() | (raw[c].str.strip() == "")).sum()) for c in
              ["CustomerName", "City", "Product", "OrderStatus"]}
print("\nLabels filled with 'Unknown' (originally blank):", label_fill)

# ---------- 7. Outliers ----------
header("7. OUTLIERS FLAGGED (and capped)")
print(df[["UnitPrice_outlier", "Quantity_outlier", "Age_outlier"]].sum().to_string())

# ---------- 8. Before vs after ----------
header("8. UNIT PRICE: BEFORE VS AFTER")
compare = pd.DataFrame({"before": prices_raw.describe(),
                        "after":  df["UnitPrice"].describe()}).round(2)
print(compare.to_string())

header("9. FINAL RESULT")
print(f"Final shape: {df[['OrderID','CustomerName','City','Product','Quantity','UnitPrice','OrderDate','OrderStatus','CustomerAg
print(f"Total amount (sum of TotalAmount): {df['TotalAmount'].sum():,.2f}")
print(f"Age range: {df['CustomerAge'].min()} to {df['CustomerAge'].max()}")
print(f"Quantity range: {df['Quantity'].min()} to {df['Quantity'].max()}")
```

```
============================================================
4. TYPE ERRORS AND BUSINESS-RULE VIOLATIONS (raw data)
============================================================
     Column  Stored as text  Rule violations
   Quantity               2                1
  UnitPrice               2                0
CustomerAge               2                1

Placeholder or implausible prices (>= 999999 or < 1): 2

============================================================
5. DATE PROBLEMS
============================================================
Dates not in YYYY-MM-DD format:        4
  of those, unreadable (set to blank): 1
Dates originally blank:                2
Dates missing in clean data:           3

============================================================
6. VALUES IMPUTED (filled with a median)
============================================================
     Column  Originally blank  Total imputed  Invalid, turned blank
   Quantity                 2              3                      1
  UnitPrice                 2              5                      3
CustomerAge                 2              3                      1

(Small differences can occur because the 5 duplicate rows were removed.)

Labels filled with 'Unknown' (originally blank): {'CustomerName': 2, 'City': 2, 'Product': 0, 'OrderStatus': 2}

============================================================
7. OUTLIERS FLAGGED (and capped)
============================================================
UnitPrice_outlier    4
Quantity_outlier     1
Age_outlier          0

============================================================
8. UNIT PRICE: BEFORE VS AFTER
============================================================
          before     after
count     182.00    180.00
mean    19407.58  14573.99
std     74639.82  15322.98
min         0.01    723.30
25%      2459.10   2498.28
50%     10451.14  11061.24
75%     19373.16  19521.74
max    999999.00  62277.55

============================================================
9. FINAL RESULT
============================================================
Final shape: (180, 10)
Total amount (sum of TotalAmount): 7,856,623.02
Age range: 3 to 70
Quantity range: 1 to 7
```

```
from IPython.display import HTML
HTML('<div style="page-break-before: always; break-before: page;"></div>')
```

Summary and Explanations

Cleaning the Messy Sales Data: Summary of Steps

I started with `Messy_data.csv` (185 rows, 9 columns) and finished with `cleaned_sales_dataset.csv` (180 rows, 10 columns). The extra column is `TotalAmount` (Quantity × UnitPrice). I also saved a second file, `cleaned_sales_dataset_with_flags.csv` (17 columns), that records every outlier and imputed value.

---

1. Loading and Inspecting the Data

I loaded the file with Pandas, reading every column as text (`dtype=str`) so nothing converted silently. Then I inspected it with `head()`, `info()`, `shape` and `isnull().sum()`. The audit showed these problems:

Issue	What I found
Missing values	2 each in CustomerName, City, Quantity, UnitPrice, OrderDate, OrderStatus and CustomerAge (none in OrderID or Product)
Duplicate rows	5 exact duplicates (OrderIDs 10006, 10026, 10081, 10121, 10151 each appeared twice)
City spellings	12 spellings for 8 real cities (" delhi", "MUMBAI", "pUnE", " bengaluru")
Status spellings	8 spellings for 5 real statuses ("completed", "SHIPPED", " pending")
Numbers stored as text	"three" and "2 pcs" (Quantity), "18,500" and "unknown" (UnitPrice), "forty" and "29 years" (CustomerAge)
Impossible values	Quantity -3 (OrderID 10068), Age 185 (OrderID 10049)
Placeholder prices	999999 (OrderID 10023) and 0.01 (OrderID 10093)
Date problems	Non-standard formats (02/03/2026, 15-02-2026) and one invalid date ("not a date", OrderID 10099)

2. Cleaning Steps

1. **Spaces and capitalization.** I trimmed leading/trailing spaces, collapsed repeated spaces and applied title case to names, cities, products and statuses. Cities went from 12 spellings to 8, and statuses from 8 to 5. I also mapped known variants (Bombay → Mumbai, Bangalore → Bengaluru, Gurgaon → Gurugram).
2. **Data types.** A custom function converted the text numbers: "three" → 3, "2 pcs" → 2, "18,500" → 18500, "forty" → 40 and "29 years" → 29. "unknown" could not be converted, so it became missing. OrderID is now an integer, Quantity and CustomerAge are whole numbers, UnitPrice is a decimal and OrderDate is a datetime.
3. **Dates.** I parsed YYYY-MM-DD dates first, then read the rest day-first, because 15-02-2026 only makes sense that way. So 02/03/2026 became 2 March 2026. The invalid date was set to missing.
4. **Business rules.** I set these to missing *before* filling any gaps, so they could not distort the medians: Quantity below 1 (1 row), age outside 0-120 (1 row) and placeholder prices of 999999 or below 1 (2 rows). No prices were zero or negative, and no dates were in the future.
5. **Duplicates.** I inspected the 10 rows sharing an OrderID, then removed **5 duplicate rows** (185 → 180), which matches the project brief.
6. **Outliers.** See Section 4.
7. **Missing values.** See Section 3.
8. **TotalAmount.** I created it as Quantity × UnitPrice (rounded to 2 decimals) and added an optional `UnitPrice_log` column to reduce skew.
9. **Region merge.** I joined a city-to-region lookup table using `merge(how="left")` and calculated sales by region.
10. **Export.** I saved the cleaned data as `cleaned_sales_dataset.csv`.

3. How I Handled Missing Values

Numeric columns were filled with the **median**, not the mean (a few extreme values can drag a mean around) and not zero (a missing price is not a price of 0). Prices used the median of the *same product*, since a laptop and a mouse should not get the same fill value.

Column	Originally blank	Invalid, set to missing	Total filled
Quantity	2	1 (the -3)	3
UnitPrice	2	3 ("unknown", 999999, 0.01)	5
CustomerAge	2	1 (the 185)	3

Text columns were labeled **"Unknown"** instead of guessed: 2 names, 2 cities and 2 statuses (Product had none missing).

Order dates were left blank on purpose. Three dates are missing: 2 were blank in the original file and 1 was invalid. Making up a date would mean making up data.

Every filled value is recorded in the `Quantity_imputed`, `UnitPrice_imputed` and `CustomerAge_imputed` columns.

4. How I Handled Outliers

I used the **IQR method**: a value is an outlier if it falls below Q1 − 1.5×IQR or above Q3 + 1.5×IQR. Prices were compared **within each product**, since a high price for a laptop is normal but the same price for a mouse is not. Quantity and age used one column-wide fence.

First, the clearly wrong values (age 185, quantity -3, prices of 999999 and 0.01) were treated as **errors** by the business rules and filled with medians. Then the remaining unusual values were **flagged and capped** to the IQR limits, so no orders were deleted:

OrderID	Product	Original price	Capped to
10007	Headphones	2,159.37	2,169.07
10016	Laptop	18,500.00	37,801.00
10110	Laptop	709.84	37,801.00
10141	Phone	815.82	11,601.91

All four prices were unusually *low* for their product, so capping raised them to the lower limit. One quantity was also flagged and capped. No ages were flagged, because the only unrealistic age (185) had already been removed. The `UnitPrice_outlier`, `Quantity_outlier` and `Age_outlier` columns keep a record of every flagged row.

5. Results

Check	Before	After
Rows	185	180
Duplicate rows	5	0
City spellings	12	8 (plus "Unknown")
Status spellings	8	5 (plus "Unknown")
Missing cells (excluding dates)	12 across 7 columns, plus 3 invalid values turned missing	0
Missing dates	2 blank + 1 invalid	3, left blank on purpose
Highest unit price	999,999	Placeholder removed and replaced by a product median

Final checks all passed: OrderIDs are unique, quantities are at least 1, prices are above 0, ages are between 0 and 120, and City has no stray spaces.

Sales by region (sum of TotalAmount):

Region	Total sales
South	2,266,993.54
West	2,068,456.60
North	2,018,414.47
East	1,426,238.33
Unknown (missing city)	76,520.08

6. Assumptions

- Dates not in YYYY-MM-DD are day-first (02/03/2026 = 2 March 2026).
- Negative quantities are data-entry errors, not product returns. A real business would want to confirm that.
- Prices of 999999 or below 1 are placeholders or entry errors.
- Capping changes the original values of the 4 price outliers. The flag columns keep that history.
- Median filling gives reasonable estimates, not true values, which is why filled rows are flagged.

7. Files Submitted

- `Sales_Data_Cleaning_Mini_Project.ipynb`: the Colab notebook
- `cleaned_sales_dataset.csv`: the cleaned data (10 columns)
- `cleaned_sales_dataset_with_flags.csv`: the same data plus outlier and imputation flags